

Auditoria Técnica

Análise Completa da Plataforma de Gestão

Data: 25 de Março de 2026

Gerado automaticamente por Lovable

1. Forças e Estrutura

1.1 *Arquitectura modular por domínio*

Os hooks de dados seguem um padrão domain-driven consistente e bem implementado. Cada área de negócio tem o seu hook dedicado que encapsula queries, mutations e lógica derivada:

- **useFinancialData** — centraliza entradas, saídas, payroll, subscrições, documentos e contractors numa única interface
- **useCommercialData** — vendas, metas mensais/trimestrais/anuais, ações e análise mensal com filtro por ano
- **useTeamData** — membros, contratos, férias, avaliações com mutations integradas
- **usePlanningData** — 14 fontes de dados automáticas para alimentar objectivos e tracking
- **useCrmData** — leads, pipelines, stages e interações com drag-and-drop integrado

Impacto: Qualquer developer novo consegue encontrar e modificar funcionalidade de um domínio sem tocar nos restantes. A separação reduz conflitos de merge e facilita testes.

1.2 *White-labeling dinâmico*

O sistema de branding em **useBusinessSettings** vai além de trocar um logo — recalcula todo o tema visual em runtime:

- Converte cores HEX para HSL e injeta-as como CSS custom properties
- Calcula automaticamente contraste de foreground (preto/branco) baseado em luminância
- Suporta fontes display e body configuráveis via Google Fonts
- Aplica-se ao login, sidebar, portal de clientes e todas as páginas internas

1.3 *Sistema de permissões granular*

A arquitectura de permissões é multicamada e bem pensada:

- **custom_roles + role_page_access**: permissões por página e por role
- **useSensitiveAccess**: controlo de acesso a dados financeiros sensíveis com password
- **usePermissions**: hook centralizado que resolve permissões do utilizador actual
- RLS policies no backend que reforçam o que o frontend permite

1.4 *Sistema de planeamento inteligente*

O **usePlanningData** é provavelmente a peça mais sofisticada do sistema. Agrega automaticamente dados de 14 fontes diferentes — vendas, tarefas, clientes, leads, conteúdos, projectos — e alimenta os objectivos e metas com dados reais em tempo real, eliminando a necessidade de input manual de progresso.

1.5 *Infraestrutura de notificações*

Sistema completo com hooks especializados:

- **useBirthdayNotifications** — aniversários de membros e clientes

- useClientRenewalNotifications — renovações de contratos
- useContractExpiryNotifications — contratos a expirar em 30 dias
- useLeadFollowupNotifications — follow-ups de CRM pendentes
- NotificationBell — centro de notificações com mark-as-read

2. Falhas e Oportunidades de Melhoria

2.1 Componentes monolíticos (CRÍTICO)

Vários ficheiros excedem largamente o tamanho recomendável:

Ficheiro	Linhas (est.)	Problema
Secretaria.tsx	~1700+	12+ hooks, múltiplos tabs inline
ExecutiveGestaoEquipa.tsx	~2410	Recentemente refactored mas ainda grande
AppSidebar.tsx	~800+	Navegação + lógica + rendering
FinMensal.tsx	~600+	Múltiplas tabelas e cálculos inline

Impacto: Tempos de re-render elevados, dificuldade de manutenção, risco de bugs em cascata ao editar uma secção.

2.2 Type Safety insuficiente (ALTO)

Uso extensivo de **as any** e tipos genéricos onde existem tipos gerados automaticamente pelo Supabase. Exemplo: queries que retornam **(data || []) as any[]** em vez de usar **Tables<'table_name'>**. Isto anula o valor do sistema de tipos do TypeScript e permite bugs silenciosos.

2.3 Ausência de paginação (ALTO)

Nenhuma query usa **.range(from, to)**. Com o limite default de 1000 rows do Supabase, qualquer tabela que cresça além disso vai silenciosamente perder dados. Tabelas em risco: commercial_sales, tasks, crm_leads, content_items, notifications.

2.4 Lógica de negócio no frontend (MÉDIO)

Processos que deviam correr no backend estão em useEffect no cliente:

- Criação de notificações de contratos a expirar
- Verificação de aniversários
- Actualização de status de vendas overdue

Problema: Se o utilizador não abrir a app, as notificações não são criadas. Múltiplos utilizadores abertos criam duplicados.

2.5 Queries N+1 (MÉDIO)

O AppLayout carrega múltiplos hooks independentes no mount. Cada hook faz as suas queries separadamente, sem batching ou prefetching coordenado. Em páginas como a Secretaria ou Executive, isto pode resultar em 15-20 queries paralelas no load inicial.

3. Lista de Melhorias Prioritárias

3.1 Refactor de componentes monolíticos **[CRÍTICO]**

Dividir Secretaria.tsx em sub-componentes lazy-loaded por tab. Cada tab passa a ser um componente independente com o seu próprio ficheiro, carregado com React.lazy(). Reduz bundle inicial e isola responsabilidades.

3.2 Tipagem estrita com tipos gerados **[ALTO]**

Substituir todas as ocorrências de 'as any' por tipos do ficheiro types.ts gerado. Usar Tables<'table_name'> para rows, Insert para inserts, Update para updates. Isto activa autocomplete e previne bugs em compile-time.

3.3 Paginação em queries críticas **[ALTO]**

Implementar .range(from, to) com infinite scroll ou paginação clássica nas tabelas de vendas, tarefas, leads e notificações. Começar pelas tabelas com mais dados.

3.4 Mover automações para Edge Functions **[ALTO]**

Criar cron jobs com pg_cron para: notificações de contrato, aniversários, status overdue de vendas, e follow-up reminders. Elimina dependência de o utilizador ter a app aberta.

3.5 Query batching e prefetching **[MÉDIO]**

Usar queryClient.prefetchQuery() para dados que sabemos que vão ser necessários. Agrupar queries relacionadas num único hook por página em vez de carregar tudo no AppLayout.

3.6 Code splitting por rota **[MÉDIO]**

Todas as páginas já usam React.lazy() nas rotas, mas os componentes pesados dentro das páginas não. Aplicar lazy loading a tabs e secções que não são visíveis no load inicial.

3.7 Testes automatizados **[MÉDIO]**

Actualmente existe apenas um test de exemplo. Adicionar testes para: hooks de dados (mock Supabase), cálculos financeiros, lógica de permissões, e fluxos críticos de CRUD.

4. Mapeamento de Automações

4.1 Automações existentes

Automação	Trigger	Resultado	Localização
Notif. contratos	Mount + query	Cria notificação se contrato expira em 30d	useContractExpiryNotifications
Notif. aniversários	Mount + query	Alerta de aniversário de membro/cliente	useBirthdayNotifications
Notif. renovações	Mount + query	Alerta de renovação de cliente	useClientRenewalNotifications
Notif. follow-ups	Mount + query	Alerta de follow-up CRM pendente	useLeadFollowupNotifications
NPS tasks	Edge Function cron	Cria tarefas de NPS para clientes	check-nps-tasks
Renewal status	Edge Function cron	Verifica status de renovações	check-renewal-status
Digest email	Edge Function	Envia resumo por email	send-digest
Welcome email	Edge Function	Email de boas-vindas a membro	create-member
Planning sources	Query reactiva	Agrega 14 fontes para objectivos	usePlanningData

4.2 Automações em falta (sugeridas)

- **Cron de status overdue** — vendas com payment_date ultrapassada deviam ser marcadas automaticamente no backend, não no frontend
- **Relatório financeiro mensal automático** — gerar PDF de resumo mensal e guardar em documentos no dia 1 de cada mês
- **Alertas de capacidade** — quando a ocupação da equipa ultrapassa 85%, notificar automaticamente
- **Onboarding automático de cliente** — ao criar cliente, popular automaticamente as actividades de onboarding com base no template do produto
- **Backup de dados periódico** — export automático de dados críticos para storage

5. Escalabilidade e Replicabilidade

5.1 Modelo actual: Instance-per-tenant

O sistema não tem coluna `tenant_id` em nenhuma tabela, nem RLS de tenant isolation. O modelo actual funciona como uma instância dedicada por negócio. Para escalar como produto, existem dois caminhos:

Opção A: Manter instance-per-tenant (Recomendado a curto prazo)

- Cada cliente tem a sua instância Lovable Cloud separada
- Isolamento total de dados — zero risco de data leaks
- Edge Function reset-instance já existe para provisioning
- Limitação: gestão de N instâncias aumenta overhead operacional

Opção B: Multi-tenant com tenant_id (Longo prazo)

- Adicionar `tenant_id` a todas as tabelas
- RLS policies com tenant isolation
- Um único deployment serve todos os clientes
- Requer refactor significativo mas escala melhor

5.2 Performance

Com paginação implementada e queries optimizadas, o sistema aguenta centenas de utilizadores simultâneos por instância. O bottleneck actual não é performance — é a ausência de paginação que vai causar problemas primeiro.

6. Os 5 Pilares do Sistema

Identificados a partir do código e da arquitectura — não do que foi declarado, mas do que está construído:

Pilar 1: Visibilidade Operacional Total

Cada área de negócio tem dashboards, métricas e análise mensal. O sistema foi construído para que nenhum aspecto do negócio fique invisível — desde vendas a tarefas, de clientes a conteúdos. O planning system com 14 fontes automáticas é a expressão máxima deste pilar.

Pilar 2: Controlo Financeiro Integrado

O módulo financeiro não é um add-on — é um dos pilares centrais. Entradas, saídas, IVA, Segurança Social, payroll, previsibilidade, balanço. Tudo ligado às vendas e aos clientes. A profundidade deste módulo revela que o controlo financeiro é um diferenciador do produto.

Pilar 3: Gestão de Pessoas como Prioridade

O sistema de team management é extraordinariamente detalhado: contratos, férias, ausências com cobertura, avaliações, escalas, work areas, responsabilidades unificadas de 8 fontes. Isto não é um módulo de RH — é um sistema de gestão de pessoas completo.

Pilar 4: Relacionamento com Cliente como Processo

Desde o CRM com pipelines customizáveis, passando pelo onboarding/offboarding estruturado, NPS automatizado, portal do cliente, até ao histórico e feedback — o ciclo de vida do cliente é tratado como um processo end-to-end, não como uma lista de contactos.

Pilar 5: Adaptabilidade por Design

White-labeling, permissões por role, temas dinâmicos, custom views no CRM, templates de deliverables, colunas customizáveis no business plan. O sistema foi construído para se adaptar a diferentes negócios, não para impor uma forma de trabalhar.